Pothole Detection using Machine Learning

A project report submitted in partial fulfillment of the requirements
for the award of the degree of

B.Tech. in

Computer Science Engineering

By

Vikrant Dabas (2021UCS1515)

Rakshit (2021UCS1533)

Veenit Kumar (2021UCS1548)

Under the supervision of

Dr. Veenu

Associate Professor

Dept of CSE

NSUT, New Delhi

DIVISION OF COMPUTER SCIENCE ENGINEERING
NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY
DELHI-110078

MAY 2025

CERTIFICATE

This is to certify that the project titled **Pothole Detection using Machine Learning** is a bonafide record of the work done by

Vikrant Dabas (2021UCS1515)

Rakshit (2021UCS1533)

Veenit Kumar (2021UCS1548)

under my supervision and guidance in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science Engineering** of the **Netaji Subhas University of Technology, Delhi-110078**, during the year 2024-2025.

Their work is genuine and has not been submitted for the award of any other degree to the best of my knowledge.

DATE: 09-05-2025

Dr. Veenu

Associate Professor

Dept of CSE

NSUT, New Delhi

DECLARATION

This is to certify that the work which is being hereby presented by us in this project titled "**Pothole Detection using Machine Learning**" in partial fulfilment of the award of the Bachelor of Technology submitted at the Department of Computer Science Engineering, Netaji Subhas University of Technology, New Delhi, is a genuine account of our work carried out during the period from January 2025 to May 2025 under the guidance of Dr. Veenu, Associate Professor, Dept of CSE, NSUT, New Delhi.

The matter embodied in the project report to the best of our knowledge has not been submitted for the award of any other degree elsewhere.

DATE: 09-05-2025

Vikrant Dabas	**Rakshit**	**Veenit Kumar**
(2021UCS1515)	(2021UCS1533)	(2021UCS1548)

ACKNOWLEDGEMENT

We would like to take this opportunity to acknowledge the support of all those without whom the completion of this project would not have been possible.

We sincerely thank our mentor, **Dr. Veenu**, for her invaluable guidance, constructive criticism, and constant encouragement throughout the course of this project. The regular interactions with her provided us with deep insights into the topic and helped us evolve our ideas. We are extremely grateful to her for giving us this opportunity and supporting us at every step.

We also wish to express our heartfelt gratitude to **Netaji Subhas University of Technology** for providing us with this invaluable opportunity for research and development. We acknowledge with much appreciation the crucial role of the staff in the Department of Computer Science Engineering, who granted us access to the necessary resources, including the lab and systems, which were essential for the successful completion of this project.

Lastly, we would like to extend our deepest thanks to our parents for their unwavering support and trust in our abilities. Their encouragement during challenging times enabled us to give our best effort in this project.

TABLE OF CONTENTS

ABSTRACT

Potholes greatly endanger roadway safety because they can damage vehicles, lead to accidents, and even fatalities. They slow normal manual evaluations of road infrastructure, especially over large scales, which consumes significant time. This work aims to address these issues by proposing fully autonomous nighttime pothole detection utilizing Wavelet Transform Convolution (WTConv) into YOLOv8 to improve detection accuracy in the dark.

This specific study stems from a growing demand for reliable unattended nighttime pothole detection systems, especially considering certain times of the day when visibility is extremely poor which increases the chance of accidents. WT-YOLOv8 answers this need by enhancing CNN-based object detection with Wavelet-Transform Convolutions (WTConv), which retain low-frequency structure while handling high-frequency noise for clearer low-light images. Combined with a PANet neck that fuses multi-scale features, the model reliably spots even small or deep, concealed potholes.

The WT-YOLOv8 architecture's main components are labeled as Backbone, Neck, and Head. The WTConv-based Backbone that extracts image detail on various frequency levels. PANet powers the Neck component that merges features of different scales, and classification and localization is performed in the Decoupled Head component which classifies objects using Binary Cross-Entropy (BCE) and regression bounding box spatial regression with Complete Intersection over Union (CIoU) for spatial regression.

In the end, our model attains mAP@0.5 = 0.906, mAP@[0.5:0.95] = 0.580, precision = 0.903, confirming both high localization fidelity and minimal false detections under severely reduced illumination. These results validate WT-YOLOv8 as a robust, real-time, and fully autonomous solution for large-scale road-surface monitoring, markedly improving roadway safety by reliably detecting hazardous potholes when human visibility is most compromised.

Chapter 1

Introduction

1.1 Pothole Detection using Machine Learning

Potholes are a significant threat to road safety, causing vehicle damage, accidents, and fatalities [1–4]. If not identified on time, their condition can worsen, and repairing them in later stages leads to economical and environmental costs [1, 5–7]. Traditional manual inspection methods are time-consuming, labor-intensive, and inefficient [8, 9]. Machine learning-based approaches offer a promising solution to automate pothole detection and improve road infrastructure maintenance. This report focuses on the development of an advanced machine learning model for nighttime pothole detection.

Machine learning techniques, particularly deep learning models, have demonstrated remarkable success in various computer vision tasks, including object detection and classification [10]. Using high-quality image datasets and state-of-the-art models, we can train systems to identify potholes accurately under challenging night conditions. The integration of artificial intelligence in road maintenance improves efficiency, reduces costs, and contributes to overall transportation safety.

1.2 Project Motivation

1.2.1 Importance of Pothole Detection at Nighttime

At night, visibility is significantly reduced and factors such as dim streetlights, headlight glare, poor weather conditions and shadows make the task of detecting potholes for the driver difficult. Thus, detecting potholes at night is critical to improving road safety [11].

Detecting potholes early allows the respective authorities to locate and repair the potholes before they deteriorate further. This leads to saving costs, logistics and the environment [7].

1.2.2 Leveraging Machine Learning for Pothole Detection

Deep learning techniques [1, 12–18] have shown remarkable success in the field of object detection. Convolutional Neural Networks (CNNs) and object detection models such as YOLO (You Only Look Once) can be used to detect potholes in low-light conditions effectively and accurately. This project aims to build a robust model tailored for night-time scenarios.

Using Wavelet Transform Convolution [19] along with this model enhances feature extraction by capturing low and high frequency image details. Thus, improving the accuracy of the model under challenging conditions.

1.3 Problem Statement

Our project focuses on addressing the challenges in nighttime pothole detection by achieving the following two key objectives:

1. **Building a Pothole Detection System**
   (a) Develop a machine learning model capable of detecting potholes in low-light conditions.
   (b) Utilizing deep learning frameworks to improve detection accuracy and efficiency.
2. **Enhancing Road Safety and Low-Light Detection**
   (a) Implement adaptive image preprocessing techniques to handle nighttime lighting variations.
   (b) Optimize model performance to minimize false positives and negatives, ensuring reliable detection.

1.4 Overview of the Thesis

1.4.1 Introduction

In this chapter, we discuss what our system represents and the theory behind why we are creating a pothole detection system, i.e., the motivation and an overview of our report chapters.

1.4.2 Literature Review

In this chapter, we have detailed the existing research and methodologies used in pothole detection using traditional and newer deep learning technologies.

1.4.3 Technical Description

In this chapter, we discuss the various technologies employed in our project. We elaborate on the techniques used for implementing the pothole detection system like the Wavelet Transform Convolution and PANet (Path Aggregation Network).

1.4.4 Methodology

In this chapter, we will look at the technologies which were used and why they were chosen of others. This will briefly explain what was needed in order to implement this project.

1.4.5 System Design and Implementation

In this chapter, we will discuss the way our system has been designed. We shall put emphasis on the various techniques that we have incorporated in our project. The various techniques in Pothole Detection System that we have implemented and how it will be used is discussed in detail.

1.4.6 Results and Analysis

In this chapter we will present the outcomes of training and evaluating the proposed object detection model, WT-YOLOv8, on the Nighttime Pothole Dataset (NPD). The evaluation employs standard COCO metrics, including mean Average Precision (mAP) at various Intersection-over-Union (IoU) thresholds.

1.4.7 Conclusions and Further Scope

In this chapter, we conclude the findings of our project and highlight the key achievements in developing a robust nighttime pothole detection system using a fine-tuned WT-YOLOv8 model. We also discuss the scope for future work, outlining potential improvements and solutions to address certain limitations in our current approach.

Chapter 2

Literature Review

This part of the report aims to summarize the work carried out in the background research period of the project development.

A key aspect of effective road maintenance and repair planning is the timely detection and assessment of potholes [20]. However, reliance on manual visual inspection for road hazard diagnosis is inefficient, costly, and labor-intensive. In recent years, multiple methods have been proposed for the automated detection and estimation of potholes. These methods are broadly categorized into two main classes: traditional methods and deep learning approaches.

2.1 Traditional Pothole Detection Methods

Traditional methods rely on vibration-based and 3D reconstruction techniques. However, they exhibit limited robustness in detecting potholes within complex environments and under varying lighting conditions.

Kim and Ryu [21] proposed an algorithm that combines vibration sensors and vehicle dynamic data to effectively identify potholes by analyzing changes in road surface vibrations detected by onboard sensors. Koch and Brilakis [22] designed an automated method for detecting asphalt road potholes using histogram-shape thresholding to segment images and geometric features to estimate potential pothole shapes. Buza et al. [23] introduced a 2D image-based pothole detection method, utilizing shape detection and Hough transforms to analyze geometric features and texture information for pothole identification.

Kim et al. [4] utilized laser scanners and stereoscopic vision techniques for precise 3D road reconstruction to facilitate pothole detection. Additionally, Ryu et al. [24] proposed an enhanced 2D image-based pothole detection method designed for intelligent transportation systems (ITSs) and road management systems. They used 2D road images collected by survey vehicles and compared their performance under various conditions such as road types, recording conditions, and lighting.

2.2 Deep Learning Methods for Pothole Detection

With the rapid advancement of deep learning, significant progress has been made in road pothole detection using deep neural networks [25–27]. These methods fully leverage the characteristics of annotated samples, effectively enhancing the precision and robustness of detection.

Due to their remarkable capability to effectively extract image features, many researchers are keen to leverage deep learning networks for enhanced pothole detection. Numerous methods use convolutional neural networks (CNNs) for feature extraction to accurately localize and detect pothole areas [13, 14].

Bhatia et al. [15] explored thermal images, which can provide different insights compared to visible spectrum images, potentially revealing potholes not detectable through standard imaging methods. Chellaswamy et al. [16] focused on a CNN-based approach for detecting both potholes and bumps, showcasing the versatility of CNNs in addressing various road anomalies. Saisree and Kumaran's [17] work emphasized classifying road conditions into discrete categories, a method useful for broader classification tasks beyond just pothole detection. Khan et al. [18] utilized YOLO for its efficiency and speed in object detection, suitable for integration into autonomous vehicle systems where real-time processing is crucial.

Chapter 3

Technical Description

3.1 Wavelet Transform Convolution

Wavelet Transform Convolution (WTConv) [19] is an advanced convolutional technique designed to address the limitations of conventional Convolutional Neural Networks (CNNs), particularly their constrained receptive fields. Traditional CNNs expand their receptive fields by increasing kernel sizes, which significantly increases the number of parameters, leading to computational inefficiency. WTConv overcomes this challenge by integrating the Wavelet Transform (WT), which enables larger receptive fields with minimal parameter growth. A key advantage of WTConv is its logarithmic increase in trainable parameters relative to kernel size, ensuring scalability and efficiency. By leveraging WT, this method effectively captures both high- and low-frequency components, improving shape bias and robustness against image distortions and corruptions.

WTConv acts as a drop-in replacement for depth-wise convolutions, seamlessly integrating into various CNN architectures such as ConvNeXt and MobileNetV2. This technique has demonstrated superior performance across multiple computer vision tasks, including image classification, semantic segmentation, and object detection. Empirical evaluations show that WTConv enhances classification accuracy and improves segmentation and detection efficiency while using fewer parameters compared to conventional CNNs. Moreover, its ability to maintain a stronger response to shapes rather than textures makes it highly effective for real-world applications where robustness to variations is essential.

3.1.1 Wavelet Transform as Convolutions

In this work, we employ the Haar Wavelet Transform (WT) [19] due to its computational efficiency and simplicity. However, this approach is not limited to Haar wavelets; other wavelet bases can be utilized, albeit with increased computational overhead. The WT operates by decomposing an image into frequency components, capturing both high and low-frequency information while maintaining spatial locality. This decomposition is particularly beneficial for CNNs, as it facilitates large receptive fields without an excessive increase in parameters.

For a given image X, the one-level Haar WT over one spatial dimension (width or height) is achieved using depth-wise convolution with the kernels:

$$\frac{1}{\sqrt{2}}[1, 1], \quad \frac{1}{\sqrt{2}}[1, -1]$$

followed by a standard downsampling operation with a factor of 2. The 2D Haar WT is implemented by applying these operations in both spatial dimensions, resulting in a depth-wise convolution with a stride of 2 using the following four filters:

$$f_{LL} = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad f_{LH} = \frac{1}{2}\begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}, f_{HL} = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}, \quad f_{HH} = \frac{1}{2}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

Here, f_{LL} acts as a low-pass filter, preserving smooth low-frequency details, while f_{LH}, f_{HL}, f_{HH} capture high-frequency components associated with horizontal, vertical, and diagonal edges, respectively. The transformation of the image X into its wavelet components is expressed as:

$$[X_{LL}, X_{LH}, X_{HL}, X_{HH}] = \text{Conv}([f_{LL}, f_{LH}, f_{HL}, f_{HH}], X) \tag{3.1}$$

where each output channel represents a different frequency band, and the spatial resolution of each component is half that of the original image. The low-frequency component X_{LL} retains the coarse structure of X, while X_{LH}, X_{HL}, X_{HH} emphasize finer details.

Since the filters form an orthonormal basis, the inverse wavelet transform (IWT) reconstructs the original image using a transposed convolution:

$$X = \text{ConvTranspose}([f_{LL}, f_{LH}, f_{HL}, f_{HH}], [X_{LL}, X_{LH}, X_{HL}, X_{HH}]) \tag{3.2}$$

This ensures perfect reconstruction of the original image from its wavelet components. By recursively applying this decomposition, the wavelet transform extends to multiple levels, where the decomposition at level i is given by:

$$[X_{LL}^{(i)}, X_{LH}^{(i)}, X_{HL}^{(i)}, X_{HH}^{(i)}] = \text{WT}(X_{LL}^{(i-1)}) \tag{3.3}$$

where $X_{LL}^{(0)} = X$ and i represents the current decomposition level. As the decomposition progresses, the spatial resolution of the low-frequency components decreases while frequency resolution increases, effectively capturing large-scale structures in the image.

This wavelet-based approach enables convolution operations in the wavelet domain, significantly increasing the receptive field without incurring high computational costs.

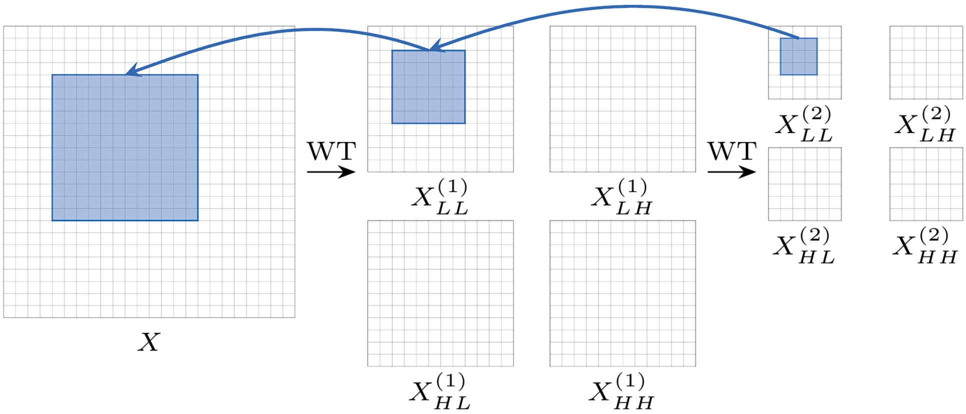

Figure 3.1: Illustration of receptive field expansion using wavelet transform. A small kernel in the wavelet domain corresponds to a much larger receptive field in the input image.

For instance, a 3×3 convolution applied to the low-frequency band of the second-level wavelet domain $X_{LL}^{(2)}$ results in a 9-parameter convolution responding to lower frequencies in a 12×12 receptive field in the input X.

3.1.2 Convolution in the Wavelet Domain

Increasing the kernel size of a convolutional layer increases the number of parameters (and, therefore, the degrees of freedom) quadratically [19]. To mitigate that, we propose the following. First, use the Wavelet Transform (WT) to filter and downscale the lower- and higher-frequency content of the input. Then, perform a small-kernel depth-wise convolution on the different frequency maps before using the Inverse Wavelet Transform (IWT) to construct the output. In other words, the process is given by:

$$Y = \text{IWT}(\text{Conv}(W, \text{WT}(X))) \tag{3.4}$$

where X is the input tensor, and W is the weight tensor of a $k \times k$ depth-wise kernel with four times as many input channels as X. This operation not only separates the convolution between the frequency components but also allows a smaller kernel to operate in a larger area of the original input, i.e., increasing its receptive field with respect to the input. See Fig. 4.1 for an illustration.

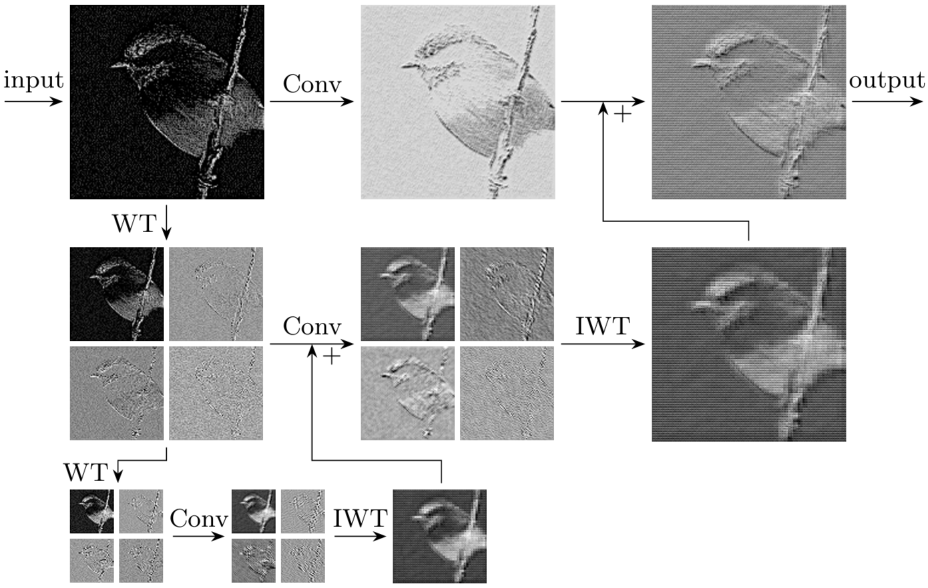

Figure 3.2: An example of the WTConv operation on a single channel taken from the third inverted residual block of MobileNetV2 using a 2-level wavelet decomposition and 3×3 kernel sizes for the convolutions.

We take this 1-level combined operation and extend it further using the same cascade principle. The process is given by:

$$(X_{LL}^{(i)}, X_{H}^{(i)}) = \text{WT}(X_{LL}^{(i-1)}) \tag{3.5}$$

$$(Y_{LL}^{(i)}, Y_{H}^{(i)}) = \text{Conv}(W^{(i)}, (X_{LL}^{(i)}, X_{H}^{(i)})) \tag{3.6}$$

where $X_{LL}^{(0)}$ is the input of the layer, and $X_{H}^{(i)}$ represents all three high-frequency maps at level i.

To combine the outputs of the different frequencies, we use the fact that the WT and its inverse are linear operations, meaning that:

$$\text{IWT}(X + Y) = \text{IWT}(X) + \text{IWT}(Y) \tag{3.7}$$

Therefore, we perform:

$$Z^{(i)} = \text{IWT}(Y_{LL}^{(i)} + Z^{(i+1)}, Y_H^{(i)}) \tag{3.8}$$

results in the summation of the different levels' convolutions, where $Z^{(i)}$ is the aggregated output from level i onward. This is in line with previous works, where two outputs of different-sized convolutions are summed as the output.

Unlike prior work, we cannot normalize each of $Y_{LL}^{(i)}, Y_H^{(i)}$ separately, as the separate normalization of these components does not correspond to normalization in the original domain. Instead, we find that performing only a channel-wise scaling is sufficient to weigh each frequency component's contribution appropriately.

3.2 PANet (Path Aggregation Network)

PANet (Path Aggregation Network) [28] is an advanced instance segmentation framework that enhances information flow through a feature pyramid network (FPN) by incorporating bottom-up path augmentation and adaptive feature pooling. It improves object localization and mask prediction while maintaining backbone independence, making it effective across various architectures.

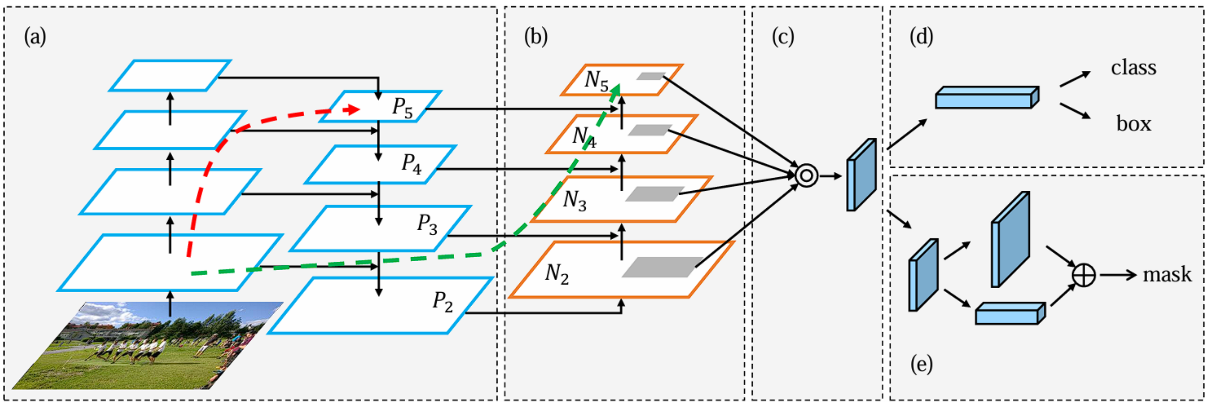

Figure 3.3: Illustration of our PANet (Path Aggregation Network)

Method:

(a) **FPN Backbone**: The Feature Pyramid Network (FPN) is utilized for multi-scale feature extraction by combining high-level semantics with low-level fine-grained details. It provides a robust feature hierarchy for effective segmentation.

(b) **Bottom-up Path Augmentation**: Enhances localization by propagating low-level strong responses to higher levels using convolution and lateral connections, integrating texture and semantic details efficiently.

(c) **Adaptive Feature Pooling**: Dynamically selects relevant features from multiple levels by pooling and fusing through element-wise operations, ensuring precise predictions across different scales.

(d) **Box Branch**: Processes enhanced feature maps to refine object localization, using fully connected layers with feature fusion to improve robustness and accuracy.

(e) **Fully-connected Fusion**: Integrates convolutional and fully connected layers for mask prediction, utilizing a hybrid approach to preserve spatial details and enhance segmentation precision.

PANet outperforms traditional Mask R-CNN with FPN, securing top rankings in the COCO 2017 Challenge for instance segmentation and object detection. Its superior performance is further validated across datasets like Cityscapes and MVD, showcasing state-of-the-art accuracy and generalization.

With a backbone-independent design, PANet is versatile and compatible with various CNN architectures. Its innovations in feature aggregation and adaptive pooling establish it as a leading framework for high-precision instance segmentation in real-world applications such as autonomous vehicles and robotics.

Chapter 4

Methodology

4.1 Proposed Model

The proposed model for Pothole Detection using Machine Learning is based on WT-YOLOv8, an enhanced version of YOLOv8, which integrates Wavelet Transform Convolution (WTConv) [19] to improve detection performance in low-light conditions. The key improvements in WT-YOLOv8 include:

- Expanded receptive field using wavelet transformation, allowing better feature extraction for pothole detection.
- Retention of low-frequency details, ensuring better visibility of potholes even under poor illumination.
- Improved detection accuracy compared to standard YOLOv8 and other object detection models.

4.2 Algorithms

Our baseline detector architecture consists of three stages: Backbone, Neck, and Head. The Backbone and Neck incorporate different algorithms, while the Head decouples tasks and applies loss functions based on bounding box regression or classification, following YOLOv8's dual-task heads. WT-YOLOv8 architecture, which integrates a wavelet transform convolution (WTConv) module [19] into the Backbone of YOLOv8. WTConv enhances the model's ability to capture both local and global features by leveraging multi-frequency response and a large receptive field. The PANet [28] Neck encodes features using a bottom-up path to combine deep and shallow features, followed by a top-down path. The prediction Head retains YOLOv8's original dual-task structure, consisting of a classification head and a regression head.

4.2.1 WTConv (Wavelet Transform Convolution) Algorithm

WTConv [19] is employed in the Backbone stage to enhance feature extraction. The Haar Wavelet Transform is used to decompose an image into low-frequency and high-frequency components:

- Low-frequency components (X_{LL}) capture essential shape and contour details.
- High-frequency components (X_{LH}, X_{HL}, X_{HH}) are filtered to reduce noise.
- WTConv applies small convolutional kernels (3×3) to frequency sub-bands and reconstructs the output using the inverse wavelet transform.

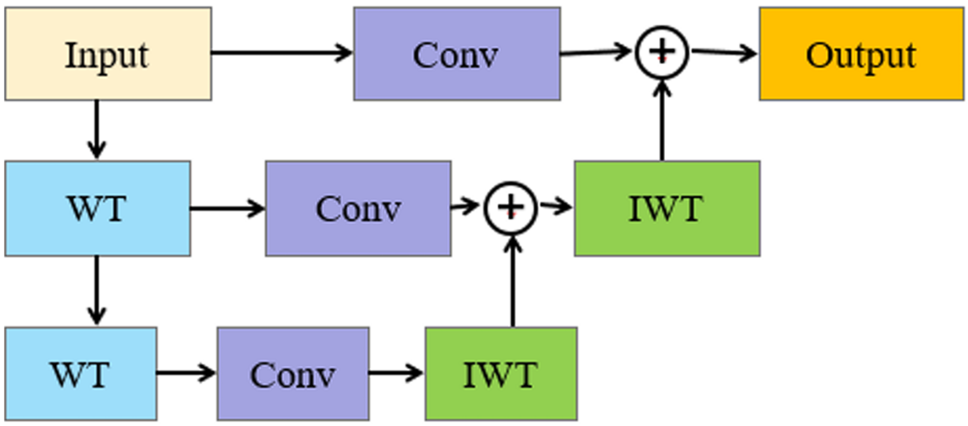

Figure 4.1: WTConv operation on a single channel using a 2-level wavelet decomposition.

4.2.2 PANet (Path Aggregation Network) Algorithm

Role: Used in the Neck stage of WT-YOLOv8 for multi-scale feature fusion.

Objective: Enhances object detection by combining high-level and low-level features, ensuring robust feature extraction at different scales [28].

Implementation:

- Bottom-up Path Augmentation: Strengthens feature propagation from low- to high-resolution layers.

- Adaptive Feature Pooling: Aggregates features across multiple scales for improved object localization.
- Top-down Feature Enhancement: Combines high-resolution and low-resolution feature maps to improve contextual understanding.

4.3 About Data

The objective is to enhance road safety by detecting potholes at night using computer vision. Potholes pose a significant risk, especially when filled with water, as they can cause skidding, loss of control, and accidents. Accurate detection reduces vehicle damage and improves safety.

One of the key challenges is the lack of high-quality annotated datasets for low-light conditions. Night images lack high-frequency details due to poor illumination, making detection difficult for traditional CNNs.

The dataset used for this study is the Nighttime Pothole Dataset (NPD) [11]. This dataset contains annotated pothole images captured under nighttime conditions.

4.3.1 Data Exploration and Insights

To ensure an efficient and structured evaluation, the Nighttime Pothole Dataset (NPD) [11] is divided into training (2625 images), validation (754 images), and test (376 images) subsets. This division helps maintain a balanced distribution of pothole characteristics, ensuring generalization across real-world scenarios.

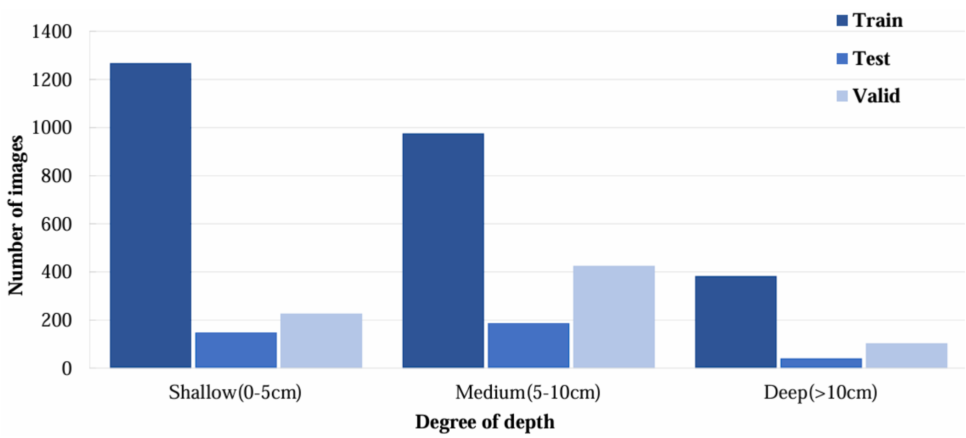

Figure 4.2: The proportion of potholes at different depths in the NPD [11].

Figure 4.2 presents the pothole depth distribution across three categories: shallow (0-5 cm), medium (5-10 cm), and deep (more than 10 cm). The training set contains 1267 shallow, 975 medium, and 383 deep potholes, while the validation and test sets include 425 and 187 medium potholes, respectively. This ensures diversity in pothole severity across all subsets, which is crucial for effective model training.

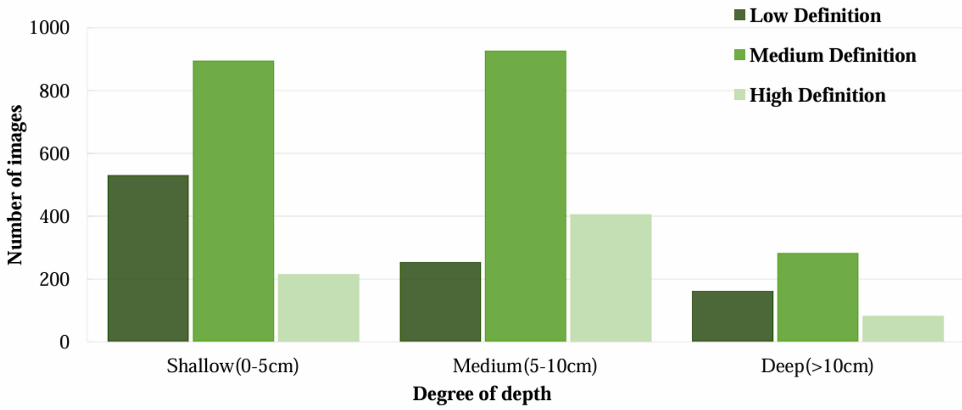

Figure 4.3: The image clarity of pothole images at different depths in the NPD [11].

Figure 4.3 analyzes image clarity, classifying images into low, medium, and high definition. The dataset consists of 531 low-definition, 895 medium-definition, and 215 high-definition shallow pothole images. Similarly, for medium-depth potholes, there are 254 low-definition, 927 medium-definition, and 406 high-definition images. Deep potholes, being more affected by poor lighting, have 162 low-definition, 283 medium-definition, and 82 high-definition images.

Nighttime conditions introduce noise and blurriness, making pothole detection more challenging. These insights emphasize the need for robust deep-learning models capable of handling low-light conditions effectively.

4.4 Language

4.4.1 Python

We have used Python as the primary programming language to develop and implement our machine learning models. Python is a high-level, interpreted, general-purpose programming language known for its simplicity, versatility, and readability. It is widely used for machine learning, data analysis, and deep learning applications due to its extensive library support and strong community backing [29].

4.4.2 Python Libraries Used

pywavelets

PyWavelets is a Python library designed for performing discrete wavelet transforms (DWT), which allow for multi-resolution analysis of signals and images [30]. In this project, PyWavelets is used to extract both high-frequency and low-frequency components from input images. These components enhance the model's ability to detect subtle pothole features, especially in challenging nighttime conditions. The integration of wavelet-transformed features helps improve detection accuracy under varying lighting scenarios.

torch

PyTorch is a popular deep learning framework that provides efficient tensor operations, dynamic computational graphs, and support for GPU acceleration [31]. It serves as the backbone for defining and training the WT-YOLOv8 model in our project. PyTorch's flexibility enabled the integration of custom wavelet convolution (WTConv) layers, which were essential for enhancing the feature extraction process. The framework also facilitated efficient training using modern optimization techniques and precision control.

ultralytics

Ultralytics is the official library that provides implementations of the YOLOv8 object detection models [32]. It offers pre-configured architectures, training pipelines, and evaluation metrics optimized for real-time performance. In this project, we leveraged the Ultralytics YOLOv8s model as the base detector and fine-tuned it for nighttime pothole detection. Modifications were made to incorporate WTConv layers into the network, allowing us to combine the power of YOLOv8's real-time detection with enhanced frequency-based feature extraction.

4.4.3 GitHub

GitHub was used as a version control platform for managing and collaborating on the project code [33]. It provided a structured and efficient way to track changes, facilitate teamwork, and maintain a well-organized repository of our datasets, model implementations, and experimental results.

Chapter 5

System Design and Implementation

Our baseline detector architecture consists of three stages: Backbone, Neck, and Head. Each stage incorporates different algorithms to optimize object detection. The Backbone and Neck handle feature extraction and fusion, while the Head decouples classification and regression tasks, applying loss functions accordingly. Our approach follows YOLOv8's dual-task heads but integrates a novel Wavelet Transform Convolution (WTConv) module [19] in the Backbone, enhancing feature extraction capabilities.

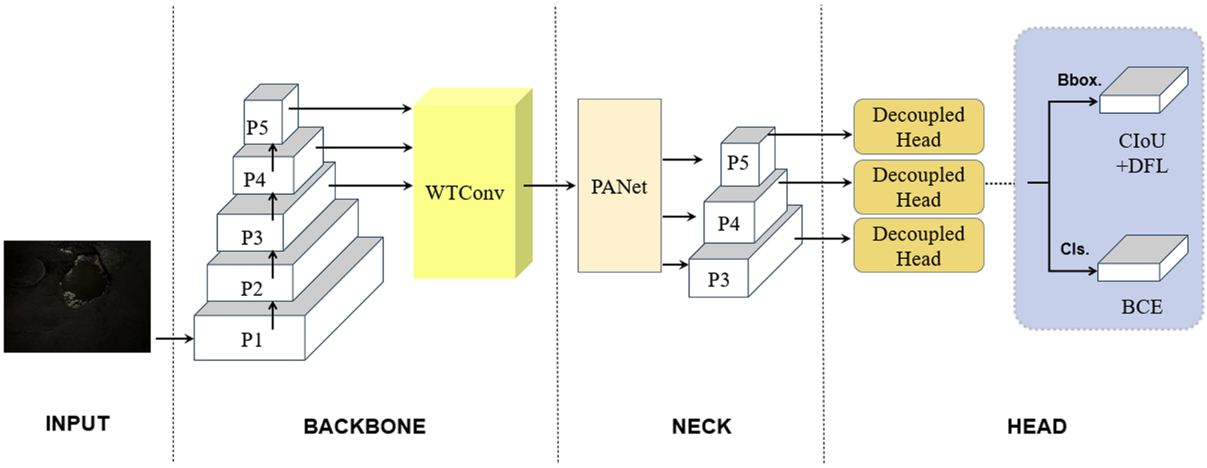

Figure 5.1: The network architecture of our baseline detector, WT-YOLOv8, which inherits its structure from YOLOv8.

5.1 Data Loading

The dataset used for this study is the Nighttime Pothole Dataset (NPD), available on GitHub [11]. This dataset contains annotated pothole images captured under nighttime conditions. The dataset is provided in COCO format, where potholes are annotated using bounding boxes. The structure of the dataset includes:

- Pre-divided dataset in COCO format.
- Pothole annotations represented as bounding boxes.
- Direct usability for object detection tasks.

5.2 Backbone

The Backbone is responsible for extracting hierarchical features from the input image. In WT-YOLOv8, we integrate WTConv (Wavelet Transform Convolution) into YOLOv8's Backbone, significantly improving the model's ability to capture both local and global features, which is particularly beneficial for nighttime pothole detection.

WTConv Algorithm

WTConv [19] applies the Haar Wavelet Transform (HWT) due to computational efficiency, decomposing the input image into low-frequency (X_{LL}) and high-frequency (X_{LH}, X_{HL}, X_{HH}) components:

- **Low-frequency components (X_{LL}):** Preserve essential shape and contour details, crucial for detecting potholes under poor illumination conditions.
- **High-frequency components (X_{LH}, X_{HL}, X_{HH}):** Filtered to reduce noise and enhance feature selectivity, addressing the challenge of high-frequency detail loss in nighttime images.

Purpose: Enhances CNN receptive field while preventing excessive parameterization.

2D Haar WT extends these operations across two dimensions, resulting in four filters: $f_{LL}, f_{LH}, f_{HL}, f_{HH}$

Process:

1. **Decomposition:** Input image X is split into low-frequency (X_{LL}) and high-frequency components (X_{LH}, X_{HL}, X_{HH}).

2. **Convolution:** Separate 3×3 convolutions are applied to each frequency sub-band, allowing better feature extraction with a larger receptive field while maintaining fewer parameters.

3. **Reconstruction:** The extracted features are merged back using the inverse wavelet transform (IWT) to maintain structured feature representation.

$$Y = IWT(Conv(w, WT(X))) \tag{5.1}$$

Where,

- X: Input tensor
- w: Convolution kernel weights
- $Conv, WT, IWT$: Convolution operator, wavelet transformation, inverse wavelet transformation

Feature Pyramid

The feature pyramid (P_1, P_2, P_3, P_4, P_5) represents different levels of extracted features:

- P_1: Captures fine-grained, low-level details.
- P_5: Extracts high-level, abstract features essential for robust object detection.

By integrating WTConv into the YOLOv8 backbone, WT-YOLOv8 effectively enhances nighttime pothole detection by focusing on low-frequency structural features, overcoming challenges posed by poor illumination.

5.3 Neck

The Path Aggregation Network (PANet) [28] is employed in the Neck to enhance feature fusion across different scales, ensuring effective multi-scale object detection. PANet consists of:

- **Bottom-up Path Augmentation:** Strengthens feature propagation from low- to high-resolution layers.

- **Adaptive Feature Pooling:** Aggregates multi-scale features for improved localization.
- **Top-down Feature Enhancement:** Merges high-resolution and low-resolution features to improve contextual understanding.

This structure allows the model to retain fine-grained details while enhancing semantic consistency across different layers. The PANet output consists of multiple feature pyramid levels (P_3, P_4, P_5), which are then passed to the next stage.

5.4 Head

The feature maps undergo Decoupled Heads, where classification and bounding box regression are processed separately. This approach improves detection accuracy by reducing feature conflicts and optimizing performance [34, 35].

5.4.1 Structure of the Decoupled Head (Inspired by YOLOv8)

Each Decoupled Head consists of:

- **Classification (Cls.) Head:** Predicts the object category using Binary Cross-Entropy (BCE) loss.
- **Regression Head:** Predicts bounding boxes (Bbox) using Complete Intersection over Union (CIoU) [36] and Distribution Focal Loss (DFL) [37] to refine object location predictions.

5.4.2 Classification Loss Function: Binary Cross-Entropy (BCE)

$$L_{cls} = -\frac{1}{\Gamma}\sum_{i=1}^{\Gamma}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right] \tag{5.2}$$

where,

- y_i represents the true label of the i-th sample (either 0 or 1)
- $\hat{y}_i$ represents the predicted value of the i-th sample (a real number between 0 and 1)
- Γ represents the number of samples.

Regression Loss (CIoU Loss)

$$L_{CIoU} = 1 - IoU + \frac{\sigma^2(m, m_{gt})}{c^2} + \lambda v \tag{5.3}$$

Where,

- IoU is the Intersection over Union
- m and m_{gt} represent center points of two bounding boxes (predicted & ground-truth)
- σ represents the Euclidean distance between these center points
- c denotes the diagonal length of the smallest enclosing box that covers both the predicted and ground-truth bounding boxes
- v measures the consistency of the aspect ratios between the two bounding boxes
- λ is a weighting coefficient.

5.4.3 Distribution Focal Loss (DFL)

$$L_{dfl} = -\left[(y_{i+1} - y)\log D_i + (y - y_i)\log D_{i+1}\right] \tag{5.4}$$

where,

$$D_i = \frac{y_{i+1} - y}{y_{i+1} - y_i} \tag{5.5}$$

$$D_{i+1} = \frac{y - y_i}{y_{i+1} - y_i} \tag{5.6}$$

Chapter 6

Results and Analysis

6.1 Overview

In the implementation, the WT-YOLOv8 model, based on YOLOv8s, was fine-tuned for night-time pothole detection using the AdamW optimizer with an initial learning rate of 0.001 and a learning rate factor (lrf) of 0.01. Training was conducted over 70 epochs with a batch size of 32 and input image resolution of 512×512 pixels. A weight decay of 0.05 was used for regularization to prevent overfitting.

For enhanced generalization, a suite of data augmentation techniques was utilized, including Mosaic, RandAugment, horizontal flipping, and random erasing with a probability of 0.4. Automatic Mixed Precision (AMP) was enabled to accelerate training and reduce memory usage. To ensure deterministic behavior and reproducibility, a fixed random seed was employed. The best model checkpoint was selected based on the highest mAP@[0.5:0.95] on the validation set. All experiments were conducted on a high-performance computing setup featuring an NVIDIA A100-SXM4-40GB GPU (CUDA:0).

6.2 Evaluation Metrics

The model's performance was primarily assessed using Average Precision (AP) and Mean Average Precision (mAP), which are standard metrics for evaluating object detection models, especially in terms of both classification and localization accuracy. For a more detailed introduction, please refer to [38].

- **Precision:** Represents the proportion of true positive pothole detections out of all predicted detections.

- **Intersection over Union (IoU):** Measures the overlap between predicted and ground truth bounding boxes, which is crucial for evaluating localization accuracy. IoU thresholds determine the criteria for considering a detection as valid.

Following the COCO evaluation metrics [39], AP was computed at multiple IoU thresholds:

- **AP@0.5:** A lenient threshold typically used in PASCAL VOC.
- **AP@[0.5:0.95]:** An average of 10 IoU thresholds from 0.50 to 0.95, used for calculating COCO-style mAP.

These metrics provide a nuanced understanding of model performance at varying levels of localization strictness, with mAP offering a comprehensive score that combines all individual AP scores.

6.3 Evaluation Results

6.3.1 Quantitative Performance on Validation Set

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      66/70      5.51G     0.8041     0.3919     0.9921         46        512: 100%|██████████| 244/244 [02:07<00:00,  1.92it/s]
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:32<00:00,  2.73s/it]
                   all        750       1404      0.896      0.848      0.903      0.575

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      67/70      5.51G     0.7966     0.3896     0.9892         40        512: 100%|██████████| 244/244 [02:08<00:00,  1.89it/s]
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:32<00:00,  2.73s/it]
                   all        750       1404      0.904      0.847      0.905      0.578

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      68/70      5.51G     0.7847     0.3846     0.9877         51        512: 100%|██████████| 244/244 [02:00<00:00,  2.03it/s]
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:21<00:00,  1.82s/it]
                   all        750       1404      0.911      0.839      0.902      0.577

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      69/70      5.51G     0.7731     0.3765     0.9789         44        512: 100%|██████████| 244/244 [01:42<00:00,  2.38it/s]
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:21<00:00,  1.77s/it]
                   all        750       1404      0.896       0.86      0.905      0.579

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      70/70      5.51G     0.7724     0.3767     0.9783         49        512: 100%|██████████| 244/244 [01:43<00:00,  2.36it/s]
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:23<00:00,  1.97s/it]
                   all        750       1404        0.9      0.856      0.905       0.58

70 epochs completed in 2.530 hours.
Optimizer stripped from runs/wt-yolov843/weights/last.pt, 22.5MB
Optimizer stripped from runs/wt-yolov843/weights/best.pt, 22.5MB

Validating runs/wt-yolov843/weights/best.pt...
Ultralytics 8.3.121 🚀Python-3.12.3 torch-2.7.0+cu126 CUDA:0 (NVIDIA A100-SXM4-40GB, 40339MiB)
Model summary (fused): 72 layers, 11,125,971 parameters, 0 gradients, 28.4 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:28<00:00,  2.36s/it]
                   all        750       1404      0.903      0.857      0.906       0.58
Speed: 0.2ms preprocess, 1.5ms inference, 0.0ms loss, 10.9ms postprocess per image
Results saved to runs/wt-yolov843
```

Figure 6.1: Model Training Output

The best-performing model checkpoint (best.pt) achieved the following results on the validation set:

Metric	Value
mAP@0.5	0.906
mAP@[0.5:0.95]	0.580
Precision (P)	0.903
Recall (R)	0.857

Table 6.1: Quantitative results on validation set

These results demonstrate the model's proficiency in detecting and localizing potholes under challenging nighttime conditions.

- **mAP@0.5:** This score indicates excellent performance in detection accuracy at a moderate IoU threshold. Values above 0.90 are generally considered very good, especially in nighttime conditions where object visibility is reduced.
- **mAP@[0.5:0.95]:** A score of 0.580 suggests the model performs well even under stricter localization conditions, which is especially important for detailed detection tasks such as pothole detection.
- **Precision:** A precision above 0.903 indicates that most of the pothole detections are accurate, minimizing false positives. This is a strong result in any object detection task.
- **Recall:** A recall of 0.857 suggests that the model is capable of detecting most of the potholes, with only a small percentage of false negatives.

6.3.2 Final Loss Values from Best Model

- **Box Loss**: This component measures how well the predicted bounding boxes align with the ground truth. A lower value indicates accurate localization.
- **Classification Loss**: Reflects how well the model classifies objects, with a lower value indicating better classification performance.
- **DFL Loss**: Distribution Focal Loss enhances bounding box regression accuracy, which is a key part of YOLOv8's architecture. This value suggests effective training in refining the model's bounding box predictions.

Loss Component	Value
Box Loss (Localization)	0.7788
Classification Loss	0.3725
DFL Loss (Reg. Accuracy)	0.9813

Table 6.2: Loss breakdown from best checkpoint

These relatively low and consistent loss values suggest the following:

(a) **Stable convergence** during training

(b) **Balanced learning** across both classification and localization tasks

6.3.3 Training Dynamics and Observations

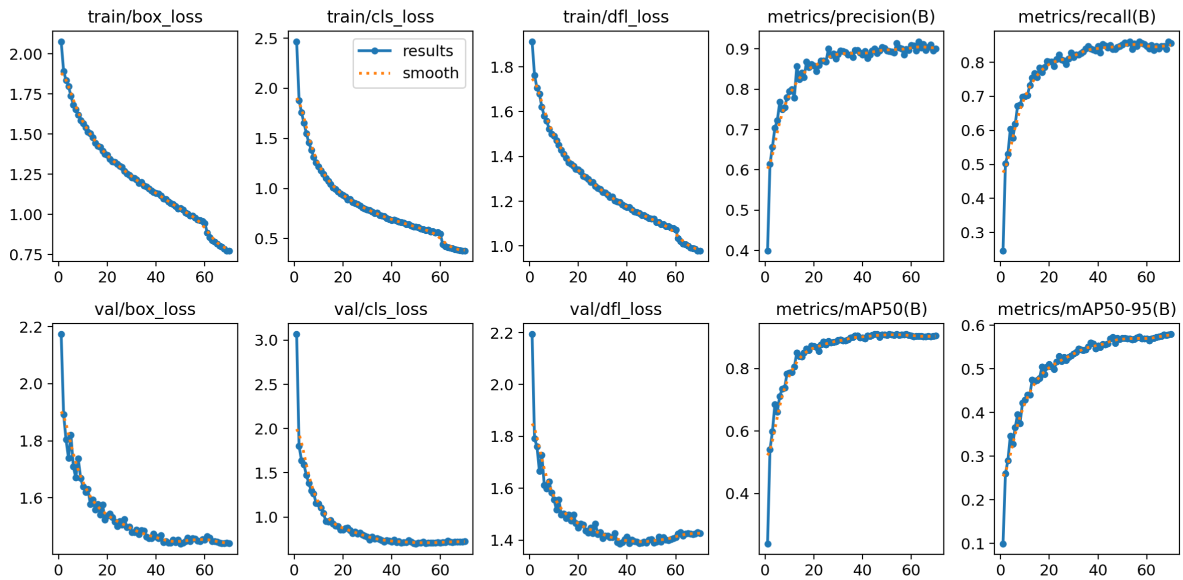

Figure 6.2: Graphs of various losses and metrics over 70 epochs

1. **Loss Curves (Training and Validation):**

    - All primary losses (box, classification, and DFL) showed a smooth decline over epochs, indicating steady progress during training.

    - The absence of oscillations or sudden spikes in the loss curves implies healthy convergence and stable learning.

2. **Precision and Recall:**

    - Precision plateaued at around 0.91, indicating that the model achieved a high rate of correct positive detections.

- Recall stabilized around 0.86–0.87, demonstrating that the model detected most potholes with minimal false negatives.

3. **mAP Performance:**

- mAP@0.5 reached approximately 0.906, showcasing excellent object localization at the lenient IoU threshold.
- mAP@[0.5:0.95] achieved 0.580, indicating a solid performance under more stringent localization criteria, though with a slight reduction in accuracy as the IoU threshold increased.

6.4 Visual Results

6.4.1 Training and Validation Batch

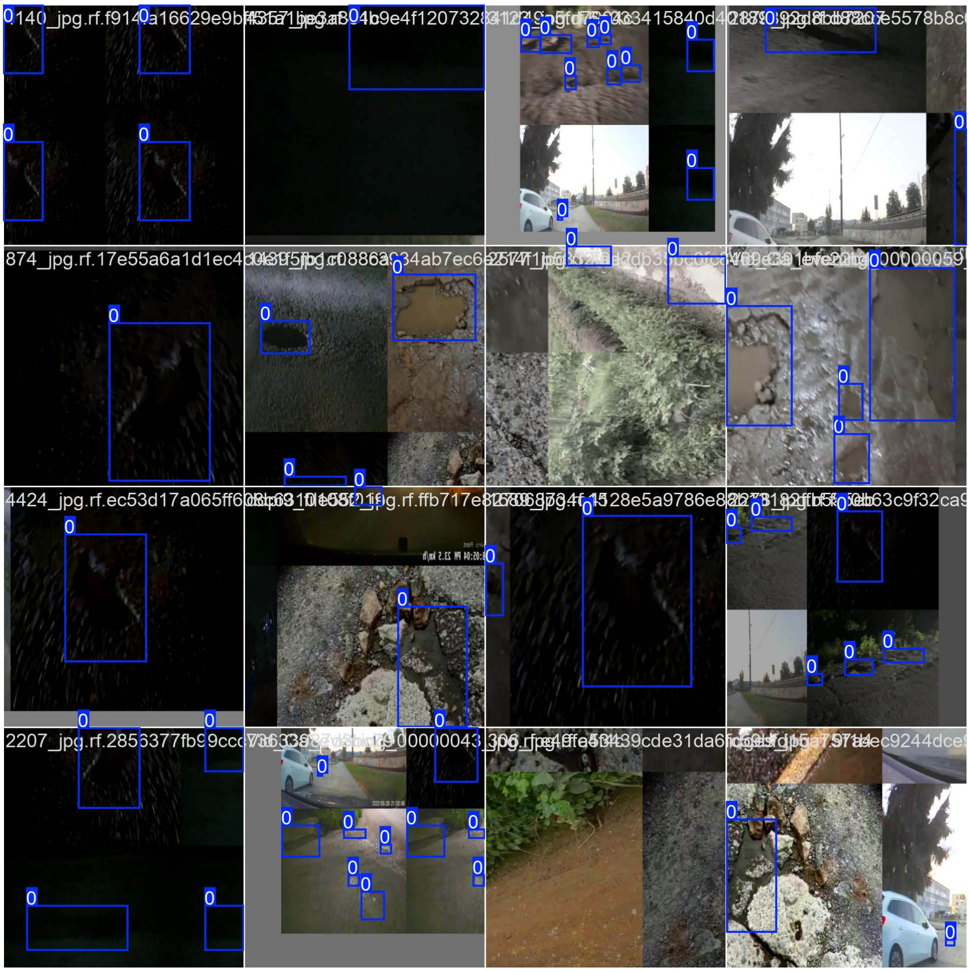

Figure 6.3: Training batch during model training

6.4.2 Comparison Between Labels and Prediction of Validation Batch

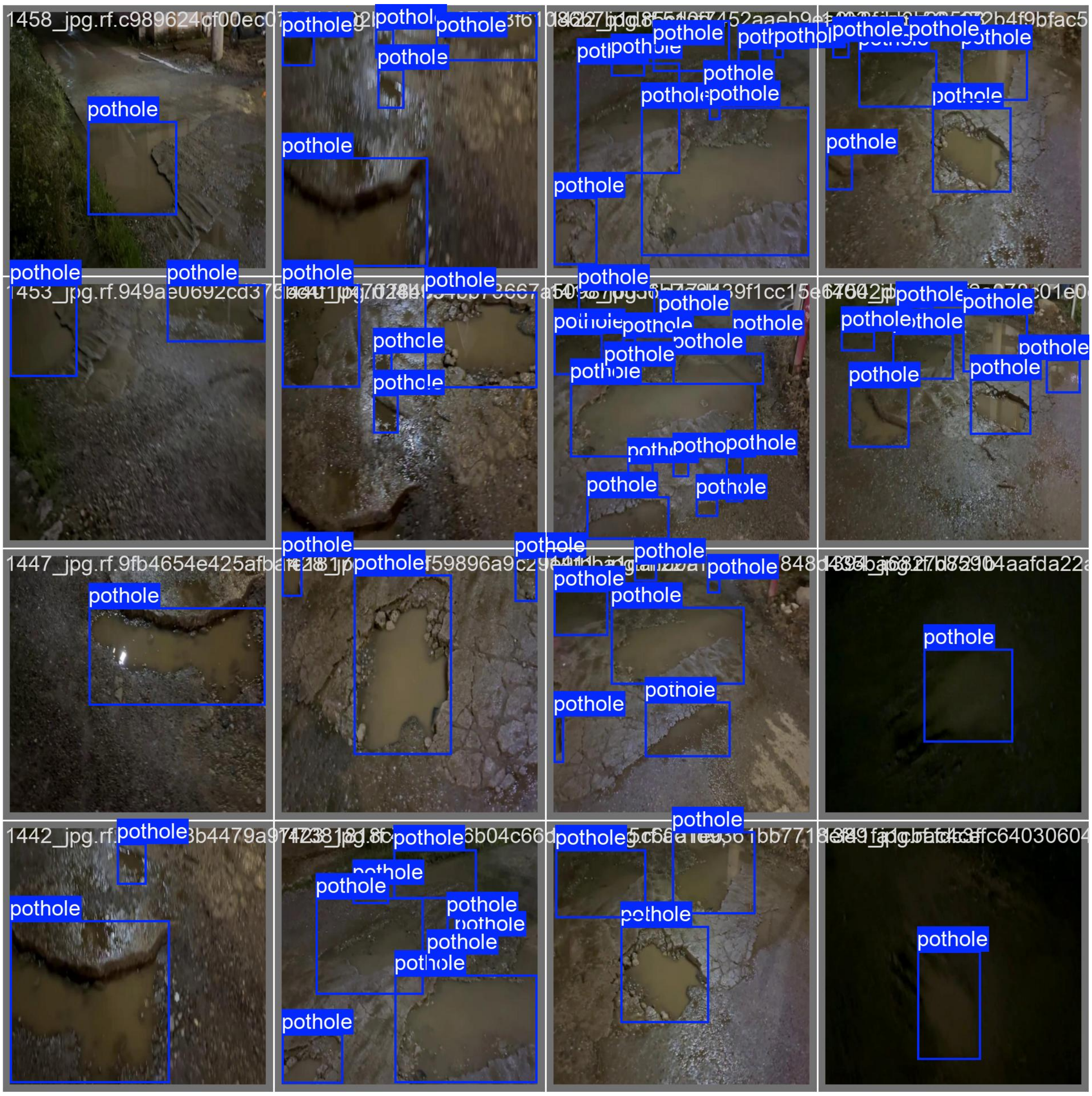

Figure 6.4: Ground truth bounding boxes on the validation set

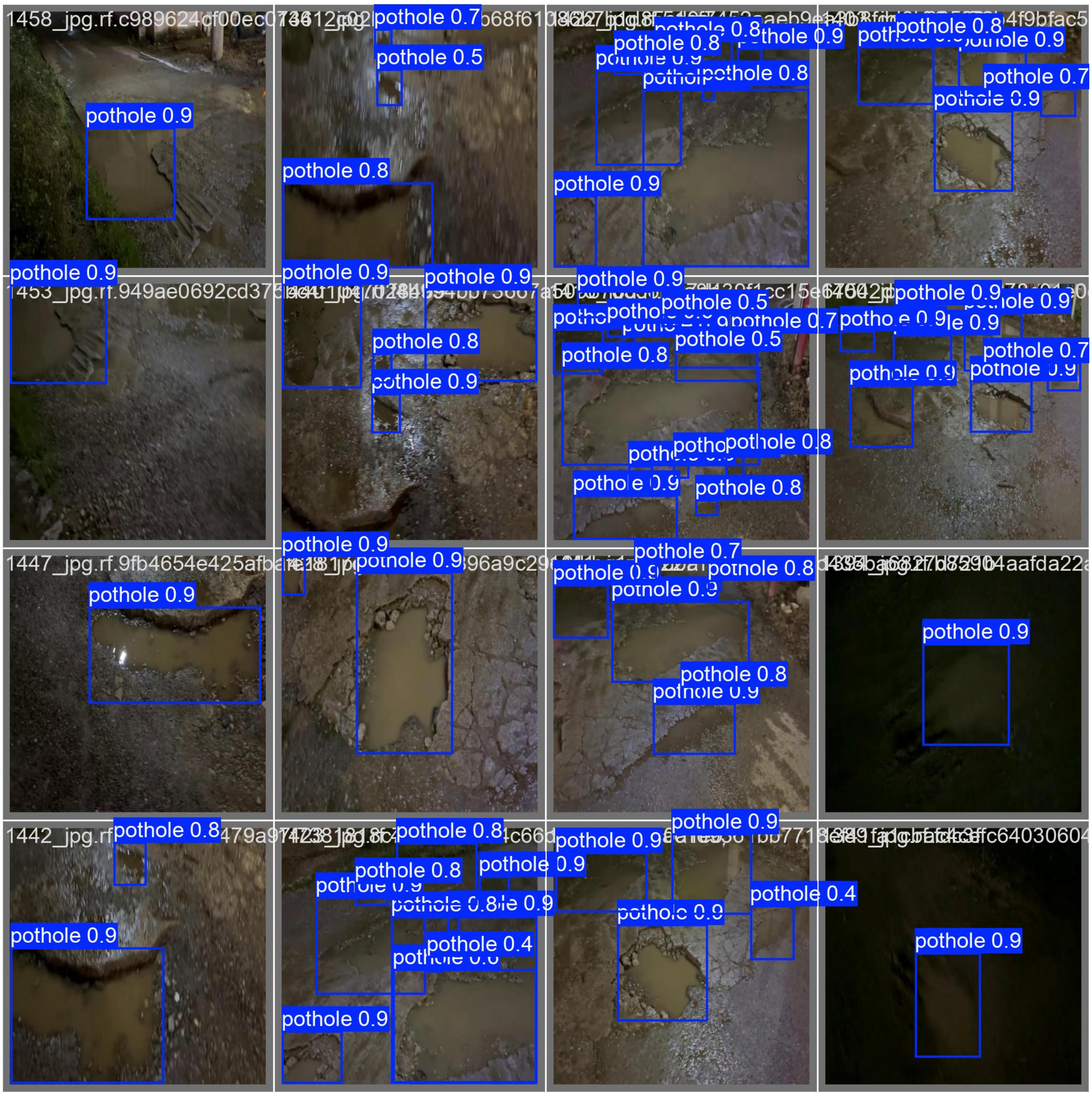

Figure 6.5: Predicted bounding boxes on the validation set

6.4.3 Final Output in Day Conditions

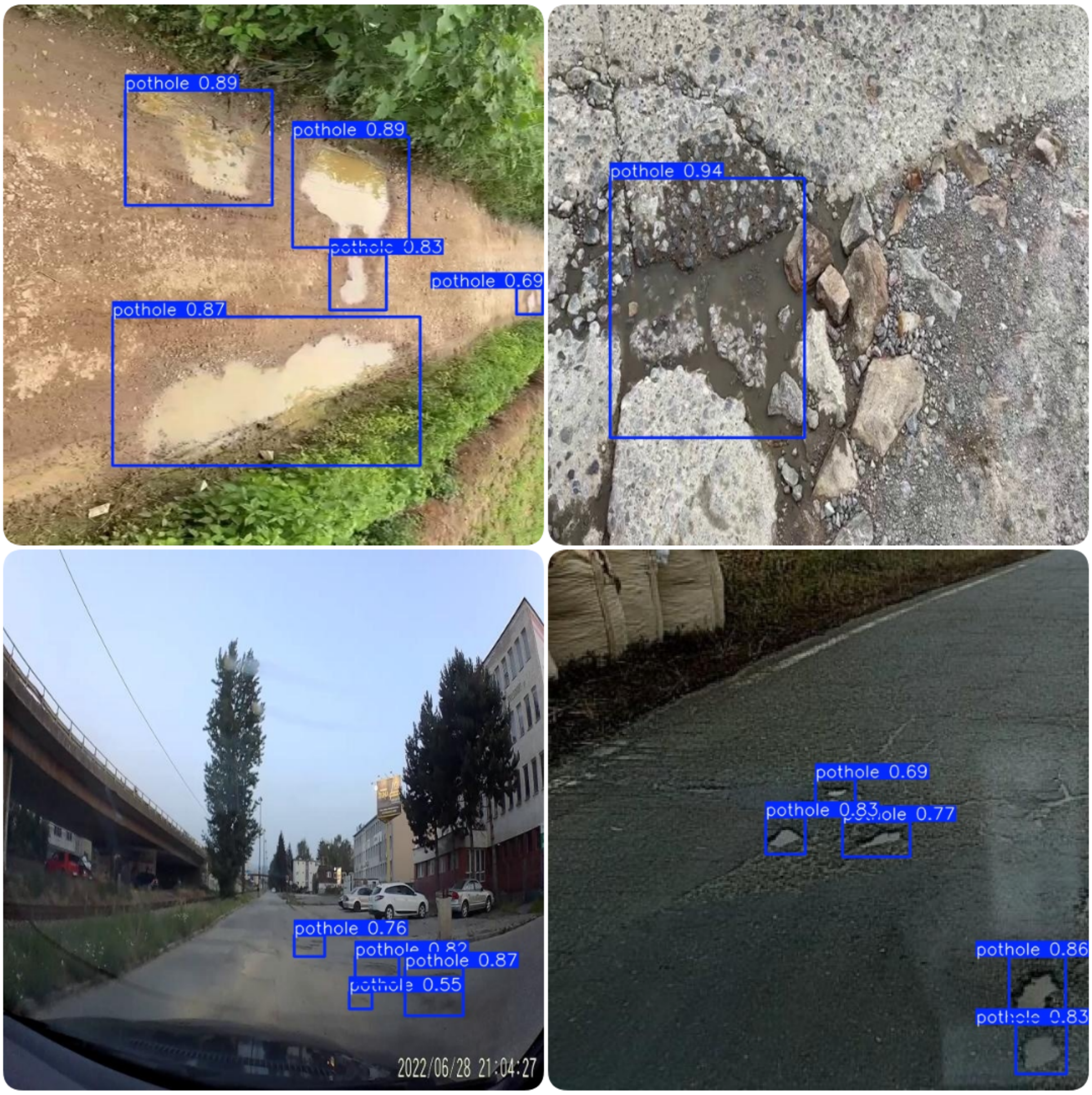

Figure 6.6: Pothole detection results in daytime conditions

6.4.4 Final Output in Night Conditions

Figure 6.7: Pothole detection results in nighttime conditions

Chapter 7

Conclusions and Further Scope

7.1 Summary

This project explored the application of a fine-tuned WT-YOLOv8 model for nighttime pothole detection, addressing the challenges posed by low visibility and environmental variability. Building upon the YOLOv8s backbone, the model was trained with an AdamW optimizer and employed a range of advanced techniques including data augmentation (Mosaic, RandAugment, random erasing), Automatic Mixed Precision (AMP), and a fixed seed for reproducibility. The model was trained for 70 epochs with a 512×512 resolution and evaluated using standard metrics like mAP@0.5, mAP@[0.5:0.95], Precision, and Recall under the COCO evaluation protocol.

The quantitative results revealed a high detection accuracy, with mAP@0.5 reaching 0.906, and mAP@[0.5:0.95] at 0.580, demonstrating the model's robustness in both lenient and strict localization settings. The loss values indicated good convergence and balanced learning across localization and classification components. Visual comparisons further validated the model's capability to perform in real-world, low-light scenarios.

7.2 Achievements

- Successfully fine-tuned a YOLOv8-based object detection model for a highly specific and challenging use case: pothole detection at night.
- Achieved state-of-the-art performance metrics for the validation set.
- Ensured stable training dynamics with no major signs of overfitting or divergence.

- Applied effective augmentation strategies and automatic mixed precision, showcasing practical deployment-readiness.
- Validated the model's performance in both day and night conditions, ensuring adaptability across lighting scenarios.

7.3 Further Improvements

While the results are promising, several directions can be explored to enhance the model further:

1. **Domain Adaptation:** Incorporate domain adaptation techniques to improve performance across different environments (e.g., rural roads, highways, or different weather conditions).
2. **Multimodal Learning:** Integrate thermal or infrared imaging to support night-time detection more robustly when RGB data quality is low.
3. **Lighter Models for Edge Deployment:** Investigate pruning and quantization techniques to make the model suitable for deployment on edge devices such as mobile phones or in-vehicle embedded systems.
4. **Larger and More Diverse Dataset:** Expand training with a more diverse dataset covering varied geographical locations and illumination scenarios (e.g., fog, streetlight glare, wet roads) to enhance the model's generalization capability. Additionally, the current dataset primarily consists of lower-resolution images, which may limit the precision of fine-grained pothole localization. Future work should consider curating or collecting a dataset with higher-quality, high-resolution images to allow the model to learn richer visual features and improve detection accuracy in real-world deployments.

Bibliography

[1] M. H. Asad, S. Khaliq, M. H. Yousaf, M. O. Ullah, and A. Ahmad, "Pothole detection using deep learning: A real-time and ai-on-the-edge perspective," *Advances in Civil Engineering*, vol. 2022, p. 9221211, 2022, [CrossRef].

[2] N. Ma, J. Fan, W. Wang, J. Wu, Y. Jiang, L. Xie, and R. Fan, "Computer vision for road imaging and pothole detection: A state-of-the-art review of systems and algorithms," *arXiv*, 2022, [CrossRef].

[3] S. B. Bhavan Kumar, S. Guhan, M. Kishore, R. Santhosh, and J. Alfred Daniel, "Deep learning approach for pothole detection—a systematic review," in *2023 2nd International Conference on Electronics and Renewable Systems (ICEARS)*, Tuticorin, India, pp. 1410–1414.

[4] Y.-M. Kim, Y.-G. Kim, S. Son, S. Y. Lim, B.-Y. Choi, and D.-H. Choi, "Review of recent automated pothole-detection methods," *Applied Sciences*, vol. 12, p. 5320, 2022, [CrossRef].

[5] A. K. Pandey, R. Iqbal, T. Maniak, C. Karyotis, S. Akuma, and V. Palade, "Convolutional neural networks for pothole detection of critical road infrastructure," *Computers & Electrical Engineering*, vol. 99, p. 107725, 2022, [CrossRef].

[6] B. Bučko, E. Lieskovská, K. Zábovská, and M. Zábovský, "Computer vision based pothole detection under challenging conditions," *Sensors*, vol. 22, p. 8878, 2022, [CrossRef].

[7] A. Lincy, D. Dhanarajan, S. Kumar, and G. Gobinath, "Road pothole detection system," in *ITM Web of Conferences*, Chapel Hill, NC, USA, 2023.

[8] R. Bharat, A. M. Ikotun, A. E. Ezugwu, L. M. Abualigah, M. Shehab, and R. Abu Zitar, “A real-time automatic pothole detection system using convolution neural networks,” *Applied Computing and Engineering*, vol. 6, pp. 879–886, 2023, [CrossRef].

[9] E. Lieskovská, M. Jakubec, B. Bučko, and K. Zábovská, “Automatic pothole detection,” *Transportation Research Procedia*, vol. 74, pp. 1164–1170, 2023, [CrossRef].

[10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[11] M. Ling, Q. Shi, X. Zhao, W. Chen, W. Wei, K. Xiao, Z. Yang, H. Zhang, S. Li, C. Lu, and Y. Zeng, “Nighttime pothole detection: A benchmark,” *Electronics*, 2024, [Nighttime Pothole Dataset (NPD): GitHub] [CrossRef].

[12] H. Chen, M. Yao, and Q. Gu, “Pothole detection using location-aware convolutional neural networks,” *International Journal of Machine Learning and Cybernetics*, vol. 11, pp. 899–911, 2020, [CrossRef].

[13] D. Saraswat, P. Bhattacharya, A. Verma, V. K. Prasad, S. Tanwar, G. Sharma, P. N. Bokoro, and R. Sharma, “Explainable ai for healthcare 5.0: Opportunities and challenges,” *IEEE Access*, vol. 10, pp. 84 486–84 517, 2022, [CrossRef].

[14] S. Tanwar, T. Ramani, and S. Tyagi, “Dimensionality reduction using pca and svd in big data: A comparative case study,” in *Future Internet Technologies and Trends (ICFITT 2017) Proceedings*. Springer, Cham, 2018, pp. 116–125.

[15] Y. Bhatia, R. Rai, V. Gupta, N. Aggarwal, and A. Akula, “Convolutional neural networks based potholes detection using thermal imaging,” *Journal of King Saud University–Computer and Information Sciences*, vol. 34, pp. 578–588, 2022.

[16] C. Chellaswamy, M. Saravanan, E. Kanchana, and J. Shalini, “Deep learning based pothole detection and reporting system,” in *7th International Conference on Smart Structures and Systems (ICSSS)*, Chennai, India, pp. 1–6.

[17] C. Saisree and U. Kumaran, "Pothole detection using deep learning classification method," *Procedia Computer Science*, vol. 218, pp. 2143–2152, 2023, [CrossRef].

[18] M. Khan, M. A. Raza, G. Abbas, S. Othmen, A. Yousef, and T. A. Jumani, "Pothole detection for autonomous vehicles using deep learning: A robust and efficient solution," *Frontiers in Built Environment*, vol. 9, p. 1323792, 2024, [CrossRef].

[19] S. E. Finder, R. Amoyal, E. Treister, and O. Freifeld, "Wavelet convolutions for large receptive fields," 2024, [CrossRef].

[20] Q. Mei, M. Gül, and M. R. Azim, "Densely connected deep neural network considering connectivity of pixels for automatic crack detection," *Automation in Construction*, vol. 110, p. 103018, 2020, [CrossRef].

[21] T. Kim and S.-K. Ryu, "Review and analysis of pothole detection methods," *Journal of Emerging Trends in Computing and Information Sciences*, vol. 5, pp. 603–608, 2014.

[22] C. Koch and I. Brilakis, "Pothole detection in asphalt pavement images," *Advanced Engineering Informatics*, vol. 25, pp. 507–515, 2011, [CrossRef].

[23] E. Buza, S. Omanovic, and A. Huseinovic, "Pothole detection with image processing and spectral clustering," in *2nd International Conference on Information Technology and Computer Networks*, Qinghai, China, p. 4853.

[24] S.-K. Ryu, T. Kim, and Y.-R. Kim, "Image-based pothole detection system for its service and road management system," *Mathematical Problems in Engineering*, vol. 2015, p. 968361, 2015, [CrossRef].

[25] J.-E. Peralta-López, J.-A. Morales-Viscaya, D. Lázaro-Mata, M.-J. Villaseñor-Aguilar, J. Prado-Olivarez, F.-J. Pérez-Pinal, J.-A. Padilla-Medina, J.-J. Martínez-Nolasco, and A.-I. Barranco-Gutiérrez, "Speed bump and pothole detection using deep neural network with images captured through zed camera," *Applied Sciences*, vol. 13, p. 8349, 2023, [CrossRef].

[26] Y.-T. Huang, M. R. Jahanshahi, F. Shen, and T. G. Mondal, "Deep learning–based autonomous road condition assessment leveraging inexpensive rgb and depth sensors and heterogeneous data fusion: Pothole detection and quantification," *Journal of Transportation Engineering, Part B: Pavements*, vol. 149, p. 04023010, 2023, [CrossRef].

[27] S. Babbar and J. Bedi, "Real-time traffic, accident, and potholes detection by deep learning techniques: A modern approach for traffic management," *Neural Computing and Applications*, vol. 35, pp. 19 465–19 479, 2023, [CrossRef].

[28] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 8759–8768.

[29] P. S. Foundation, "Python language reference, version 3.11," 2023, [Online Documentation].

[30] G. R. Lee *et al.*, "Pywavelets: A python package for wavelet analysis," *Journal of Open Source Software*, vol. 4, no. 36, p. 1237, 2019, [CrossRef].

[31] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035, [PDF].

[32] Ultralytics, "Yolov8 docs: Ultralytics yolov8 models," [Online Documentation], 2023.

[33] I. GitHub, "Github docs: Version control and collaboration," [Online Documentation], 2023.

[34] Ultralytics, "Yolov8 - the latest yolo by ultralytics," 2023, [GitHub].

[35] Z. Wang and H. Liu, "Yolov8 to yolo11: A comprehensive architecture in-depth comparative review," *ResearchGate*, 2023, [CrossRef].

[36] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-iou loss: Faster and better learning for bounding box regression," *AAAI Conference on Artificial Intelligence*, 2020.

[37] Z. Li, C. Peng, G. Yu, X. Zhang, W. Deng, and J. Sun, "Distribution focal loss for dense object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 2590–2599.

[38] M. Everingham, L. Van Gool, C. Williams, J. Winn, and A. Zisserman, "The pascal visual object classes (voc) challenge," *International Journal of Computer Vision*, vol. 88, pp. 303–338, 2010, [CrossRef].

[39] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. Zitnick, "Microsoft coco: Common objects in context," in *Computer Vision – ECCV 2014: 13th European Conference*, 2014, pp. 740–755, proceedings, Part V.